

APPENDIX B

Working with Molarium: scientific workflows for humans and agents

Molarium is designed so that a scientist working through the graphical interface and an agent operating through the Chemist Actions API act on the same molecular state and encounter the same scientific boundaries. This appendix therefore describes Molarium by **scientific task**, not by software module.

Each workflow begins with the question a scientist is trying to answer. The human procedure explains how to perform the task and what to inspect before accepting the result. The accompanying **agent skill contract** expresses the same procedure as preconditions, permitted actions, evidence, success criteria, and stopping conditions. The two views are intentionally paired: a procedure should be understandable by a scientist and sufficiently explicit that an agent cannot silently change its meaning.

Implementation basis. This appendix describes the inspected browser implementation prepared on 2 September 2026 for the public Molarium release. It distinguishes current browser behavior from executable but experimental behavior. Repository-only and proposed functionality is not presented as an operating instruction. User-facing workspace names are View, Design, and Simulate; the serialized values remain **view**, **build**, and **run** so existing action scripts remain replayable.

B.1 One molecular state, two interfaces

Status. Current browser surface.

The question. What changes the molecule, and what merely changes how I am looking at it?

Molarium separates the authoritative molecular state from the interface used to manipulate it. A scientist can work in **View**, **Design**, and **Simulate**; an agent can invoke the same guarded scientific operations through the versioned Chemist Actions API. Both routes act on the same graph, coordinates, selections, calculation records, and pending chemistry transaction. Browser-native transport—choosing a local file, writing to the clipboard, downloading a file, or entering fullscreen—remains outside the molecular API; the state-changing operation before or after that boundary has a named action.

Rotating the camera, changing a representation, focusing a component, or showing contacts changes only the presentation. Loading a structure, applying a protonation state, finishing a chemistry transaction, accepting a docking candidate, running a coordinate-producing Simulate job, or selecting or playing a saved calculation frame changes the current coordinates or graph. Refined-pose and side-chain alternatives remain candidates until explicitly applied.

The common operating rule. Inspect, act, verify. Generate alternatives freely, but change the authoritative molecular state deliberately. A valid operation is not automatically a scientifically appropriate one; acceptance still depends on the question being asked.

Starting from a structure

Start with **?blank** when a reproducible workflow should begin from an empty canvas. Otherwise the application loads its bundled launch molecule. A structure can be pasted as XYZ or PDB text, uploaded as XYZ or PDB, entered as a SMILES string or PDB identifier in connected mode, or selected from the prepared examples. The current upload router should not be relied on for ordinary MOL-file ingestion, even though a V2000 parser exists elsewhere in the application.

After loading, inspect atom count, formula, formal charge, connected components, graph connectivity, and the 3D structure. For a selected small-molecule component, compare the synchronized 2D depiction with the 3D graph. For PDB input, remember that only the first model is loaded and alternate locations are resolved by the implemented occupancy policy. For XYZ input, bonds are inferred from proximity and should be checked rather than assumed.

State and persistence

Loading a new molecule resets incompatible calculation, docking, selection, and preparation state. The working session is in memory: Clear, page reload, or script import can discard unsaved work. Share copies a URL rather than serializing the current molecule, and Save produces a PNG rather than a reusable molecular record. Export the

scientific record needed by the next step before replacing or clearing the scene.

Agent skill contract: `operate_molecular_state`

Contract element	Requirement
Goal	Load and inspect a starting structure without confusing a visual change, candidate, or imported approximation with an accepted molecular state.
Preconditions	A supported input or prepared example is available; connected-mode requirements are known.
Inputs	Input type and contents; intended component; optional network policy; requested inspection fields.
Procedure	Start blank when reproducibility requires it; load through a supported route; inspect graph, components, charge, and coordinates; report ambiguities before any mutation.
State mutation	A successful load replaces the current molecule and resets dependent state. View and selection operations do not alter graph chemistry.
Evidence	Source identity or input hash where available; atom/component summary; explicit warnings about inferred bonds, alternate locations, or network retrieval.
Success	The loaded graph and coordinates correspond to the intended starting object and all material ambiguities have been surfaced.
On failure	Leave the prior scene in place where the interface does so; report the parse, fetch, or format error; do not reinterpret an unsupported file silently.
Boundary	No autosave or complete-session archive; an image or share link is not a reusable molecular state.

Structure text, identifiers, and prepared examples have explicit loading actions. A local file picker only supplies bytes to `session.loadStructure`; it is not an alternate molecular loader:

```
await api.execute({action:"session.loadStructure", args:{
  format:"pdb", content:pdbText, name:"starting complex"}})
// alternatives:
await api.execute({action:"session.loadIdentifier",
  args:{value:"50VE", kind:"pdb"}})
await api.execute({action:"session.loadFixture",
  args:{fixtureId:"trp-cage"}})
await api.execute({action:"session.inspect",
  args:{scope:"all", includeCoordinates:false, maximumAtoms:500}})
await api.execute({action:"view.setDisplay",
  args:{representation:"both", showHydrogens:false,
    showPocketAtoms:true, showInteractions:true}})
```

Registered prospective routes and public structure stories retain their narrower guarded loaders (`designRoute.load` and `structureStory.load`).

B.2 Prepare a protein for local molecular design

Status. Experimental current surface.

The question. I have an experimental protein structure. What must happen before I can safely edit a ligand or run a local calculation?

A deposited PDB structure is not automatically a simulation-ready system. Atoms may be missing, alternate locations may have been chosen, protonation can be ambiguous, crystallographic waters may or may not matter, and ligands or unusual residues may exceed the supported parameterization. Molarium’s preparation workflow is deliberately conservative: it makes a bounded set of assumptions explicit, performs repairs it knows how to perform, and stops when the structure exceeds those capabilities.

Human workflow

1. Load the PDB structure and inspect the protein, ligand, waters, other components, and the region around the intended binding site before preparing anything.
2. Open **Protein Preparation**. Choose pH, histidine policy, supported missing-heavy-atom repair, ligand treatment, water treatment, and gap policy.
3. Run **Prepare structure**. Molarium constructs an internal preview that inventories residues, missing atoms, sequence gaps, ligands, waters, clashes, and invalid coordinates. Within scope, it can retrieve CCD information,

reconstruct supported missing side-chain atoms, add standard hydrogens and terminal oxygen, choose histidine states, and scan polar-hydrogen orientations.

4. An unresolved blocker stops the operation and exposes the preview for review. A blocker-free preview is immediately parameterized and applied; there is no normal second Apply step.
5. Inspect every applied change, warning, and attached audit. Download the preparation report if the decisions must survive the browser session.

What should I inspect?

- Ligand identity, connectivity, completeness, and whether it should be retained or excluded.
- Histidine and other local protonation choices near the ligand or catalytic region.
- Crystallographic waters that bridge interactions or define the local geometry.
- Missing atoms, chain breaks, unresolved regions, reconstructed geometry, and severe clashes near the region of interest.
- Whether the resulting force-field and charge assumptions match the scientific question.

Acceptance is narrower than success. A clean preview means Molarium completed its declared preparation procedure. It does not certify a production molecular-dynamics system. The implementation explicitly records `readyForProductionSimulation: false`.

Preparation blocks unsupported residues, unsupported missing atoms, incomplete retained ligands, disallowed gaps, invalid coordinates, and severe clashes. It does not currently construct missing loops or membranes, model general metal coordination, parameterize covalent ligands, or build periodic explicit-solvent boxes. Arbitrary proteins use an experimental Sage/Gasteiger route.

Agent skill contract: `prepare_protein`

Contract element	Requirement
Goal	Convert the loaded PDB into an explicitly reviewed molecular state suitable for a supported local Molarium calculation.
Preconditions	A PDB-derived structure is loaded; the intended downstream use is known; ligand and water roles can be reviewed.
Inputs	pH; histidine policy; missing-heavy-atom repair policy; ligand policy; water policy; gap policy.
Procedure	Inspect the starting structure; run preparation; review any blocking preview or the applied result; export the report when persistence matters; stop on unresolved blockers.
State mutation	Prepared graph and coordinates replace the prior state; preparation audit and parameterization are attached; one undo snapshot is created.
Evidence	Options, detected issues, repairs, warnings and blockers, preparation report, and resulting parameterization identity.
Success	The declared procedure completes without unresolved blockers and its scope matches the intended local calculation.
Failure behavior	Report the blocker and stop. Do not manufacture missing atoms, parameters, loop geometry, or unsupported chemistry.
Escalate when	The problem requires loops, membranes, metal coordination, covalent chemistry, periodic solvent, or another unsupported preparation capability.
Does not establish	Production simulation readiness or physical accuracy of the chosen downstream model.

Representative public actions are:

```
await api.execute({action:"protein.prepare", args:{
  pH:7.4, histidine:"auto", repairMissingHeavy:true,
  ligandPolicy:"ccd", waterPolicy:"crucial", gapPolicy:"block"}})
await api.execute({action:"session.inspect",
  args:{scope:"pocket", includeCoordinates:false, maximumAtoms:500}})
```

The preparation action already parameterizes and applies a blocker-free result. `protein.parameterize` is a separate alternative for an already prepared or subsequently edited complex, not the next mandatory call. The exact argument vocabulary returned by `api.describe()` is the runtime authority; this sequence illustrates the contract rather than

replacing that manifest.

B.3 Choose the protonation state you are designing

Status. Current ligand workflow; protein choices within experimental preparation.

The question. What charged and protonated species does the molecular graph actually represent?

Protonation is part of chemical identity, not a cosmetic display choice. It changes formal charge, hydrogen count, hydrogen-bonding patterns, parameterization, and often the meaning of every calculation that follows. Molarium exposes two related but distinct workflows: ligand-state enumeration through RDKit, and protein protonation decisions inside the bounded PDB-preparation workflow.

Ligand workflow

1. Load the ligand from a SMILES string or select the ligand component whose identity you intend to change.
2. Choose a pH from 0 to 14 and enumerate candidate states. RDKit applies a versioned Dimorphite-style site table, edits formal charges and hydrogens, sanitizes each result, and returns an ordered list with estimated weights.
3. Inspect each candidate as chemistry: total charge, changed atoms, tautomeric or aromatic consequences, and compatibility with known assay conditions or the bound environment.
4. Apply one state deliberately. Molarium re-embeds the chosen species, replaces the graph and coordinates, and invalidates state that depended on the former topology.

Protein workflow

For a PDB structure, set the pH and histidine policy during preparation. The implemented policies include automatic selection or explicit HID, HIE, and HIP choices, followed by inspection of polar-hydrogen orientations. Treat residues near a ligand, metal, catalytic center, or salt-bridge network as scientific decisions rather than defaults.

What the reported ligand weights mean. They are independent Henderson-Hasselbalch estimates from the implemented site model. They are not microscopic pKa predictions, coupled-site populations, or evidence for the state selected by a protein binding site.

Agent skill contract: `select_protonation_state`

Contract element	Requirement
Goal	Make the intended protonation and formal-charge state explicit before editing or calculation.
Preconditions	A valid ligand or PDB structure is loaded; the relevant pH or experimental condition is known or stated as an assumption.
Inputs	pH; candidate-state selection; for proteins, histidine policy and local residues requiring review.
Procedure	Enumerate or preview; compare candidates; inspect charge and local interactions; apply exactly one accepted state; re-inspect the resulting graph.
State mutation	Applying a ligand state replaces graph and coordinates. Protein protonation changes are committed through preparation. Prior parameterization becomes obsolete.
Evidence	pH, enumeration or preparation output, chosen state, total charge, changed sites, and stated rationale when more than one state is plausible.
Success	The authoritative graph explicitly represents the intended state and downstream methods will see the corresponding atoms and charges.
On failure	Report invalid pH, sanitization, embedding, or worker errors; do not partially apply or choose a different state merely because it runs.
Boundary	Enumeration weights do not resolve coupled microscopic states or protein-induced pKa shifts.

Ligand-state enumeration and application use the same bounded worker and graph replacement as the visible pH controls:

```
await api.execute({action:"ligand.enumerateProtonation",
  args:{pH:7.4, maximumStates:16}})
await api.execute({action:"ligand.applyProtonation", args:{index:0}})
await api.execute({action:"session.inspect",
  args:{scope:"ligand", includeCoordinates:false}})
```

Enumeration does not select a chemically correct state. The chosen index remains a reviewable scientific decision; protein protonation is carried separately within `protein.prepare`.

B.4 Edit atoms, bonds, charges, and stereochemistry in 3D

Status. Current browser surface.

The question. How do I change the chemistry without corrupting the molecular graph or losing the ability to undo the operation?

The synchronized 2D and 3D views are two ways of selecting the same graph. Chemistry-changing actions are grouped into a transaction so that a related set of edits can be validated and committed as one scientific step. Protein covalent atoms are protected from ordinary ligand editing.

Human workflow

1. Enter **Design** and select the relevant atom or bond in either view. Confirm the stable atom identity and local environment before changing it.
2. Apply the intended element replacement, bond order or aromaticity change, formal charge, explicit hydrogen, or stereochemical operation. Related operations remain in one pending chemistry transaction.
3. For coordinate-only geometry changes, select a connected path of two, three, or four atoms to set a distance, angle, or dihedral. Molarium moves the connected side defined by the graph and blocks ambiguous ring cuts.
4. Choose **Finish chemistry**. RDKit sanitizes the graph, hydrogens are reconciled, eligible local coordinates are polished, reference contacts are remapped where applicable, obsolete parameterization is invalidated, and a single undo snapshot is created.
5. If validation fails, correct the still-pending transaction or choose **Discard** to restore the pre-edit state. Use Undo/Redo only after a transaction has been committed.

What should I inspect?

- Valence, formal charge, aromaticity, hydrogen count, stereochemistry, and the intended bond topology.
- Whether the 2D depiction and 3D graph agree after sanitization.
- New close contacts, distorted local geometry, and movement outside the edited region.
- Whether topology-dependent records—parameterization, docking mapping, or calculation results—were invalidated or remapped as expected.

A synchronous mutation error restores the transaction snapshot. If the chemistry is valid but optional coordinate polishing fails, the valid graph can remain committed and the cleanup error is recorded. Fused-aromatic edits that cannot preserve the aromatic graph are rejected rather than approximated.

Distance, angle, dihedral, atom translation, and graph-chemistry controls all have named actions. A representative graph edit follows; coordinate-only edits use `geometry.setInternalCoordinate` or `geometry.translateAtoms` and remain separate from `chemistry.finish`.

Agent skill contract: `edit_molecular_graph`

Contract element	Requirement
Goal	Apply an explicit, reviewable graph or local-geometry change while preserving atomic lineage and undoability.
Preconditions	The intended editable component is identified; no unrelated transaction is pending; the target atoms or bond are unambiguous.
Inputs	Selected stable atom identifiers; edit type and value; optional connected geometry path; intended transaction boundary.
Procedure	Inspect target; open or join a transaction; apply supported edits; finish once; inspect sanitized graph and local coordinates.
State mutation	Graph edits remain provisional until Finish. Finish commits one new state and invalidates obsolete parameterization; Discard restores the starting state.
Evidence	Pre-edit identity, ordered public actions, changed atoms and bonds, sanitization outcome, cleanup outcome, and final inspection.

Continued on next page

Contract element	Requirement
Success	The final graph encodes the intended chemistry, sanitizes successfully, and passes local visual and structural checks.
On failure	Keep the transaction pending for correction or discard it. Never bypass sanitization or inject coordinates directly.
Boundary	Local graph validity does not establish synthetic feasibility, tautomer preference, binding affinity, or global conformational plausibility.

```
await api.execute({action:"view.setMode", args:{mode:"build"}})
await api.execute({action:"build.setTool", args:{tool:"select"}})
await api.execute({action:"selection.replace", args:{atomIds:[anchorId]}})
await api.execute({action:"chemistry.addAtom",
  args:{attachedToAtomId:anchorId, element:"Cl"}})
await api.execute({action:"chemistry.finish", args:{}})
await api.execute({action:"geometry.setInternalCoordinate",
  args:{atomIds:[a,b,c,d], value:172.0, moveConnected:true}})
await api.execute({action:"session.inspect",
  args:{scope:"ligand", includeCoordinates:true, maximumAtoms:200}})
```

The serialized value `build` selects the user-facing Design workspace; it is retained for action-script compatibility.

B.5 Grow an analogue by adding and positioning fragments

Status. Current Design workspace; pose-aware extensions experimental.

The question. How do I add the substituent I have in mind while preserving the useful context of the parent structure?

Fragment growth combines a graph edit with an initial 3D placement. Those are separate scientific problems: the new molecule must first be chemically correct, and its coordinates must then be a plausible starting point for relaxation or pose maintenance. Molarium’s Design workspace stages a fragment template and applies its attachment as one undoable edit. Subsequent atom, bond, charge, hydrogen, or stereochemical changes use the pending chemistry transaction described above.

Human workflow

1. If the parent is bound and its pose matters, capture the reference pose **before** beginning the topology change. Retain any explicit core or interaction choices needed for later pose maintenance.
2. Select the ligand attachment atom and stage the desired fragment or template. Confirm which atoms are new, which parent atom remains the anchor, and which bond is to be formed.
3. Apply the attachment as one undoable fragment edit. Perform any associated leaving-group, bond-order, formal-charge, hydrogen, or stereochemical changes as a subsequent pending chemistry transaction.
4. Use distance, angle, and dihedral controls to give the new group a sensible initial orientation. These controls establish a starting geometry; they do not score the binding site.
5. Finish any subsequent pending chemistry, then inspect the attachment, local valence, stereochemistry, ring geometry, clashes, and preserved parent coordinates. Relax the structure or invoke reference-guided pose maintenance as a separate step.

How should I judge the result?

A successful fragment operation means that the intended graph was built and sanitized. It does not mean that the chosen rotamer, ring conformation, or pocket orientation is preferred. For a solvent-exposed edit, local minimization may be enough to remove construction strain. For an edit that changes pocket contacts or releases a ring, use a workflow that samples alternatives and retains the relationship to the experimental reference.

Enumeration boundary. Development modules can represent bounded edit plans, but general combinatorial enumeration, library management, automated campaign selection, and automatic MCS transfer of an independently imported analogue are not current workbench workflows.

Agent skill contract: `grow_fragment`

Contract element	Requirement
Goal	Construct one intended analogue and a reviewable starting geometry while retaining parent atom lineage and, when requested, reference-pose context.
Preconditions	Editable ligand and attachment atom identified; fragment chemistry specified; reference captured first if pose continuity matters.
Inputs	Parent and attachment atom identifiers; fragment/template; new bond; associated deletions, charge or stereochemical edits; optional geometry targets.
Procedure	Capture reference if needed; stage and attach the fragment; perform and finish any associated atom/bond transaction; set only the geometry needed for a starting pose; inspect; relax or sample separately.
State mutation	Attachment commits the fragment graph and an undo snapshot immediately. Finishing a later atom/bond transaction commits those additional changes. A later pose or optimization changes coordinates separately.
Evidence	Parent and product inspection, atom mapping, edit actions, transaction outcome, initial placement rationale, and any preserved reference/contact record.
Success	The attached analogue graph is correct on inspection; any subsequent chemistry transaction is sanitized; the initial geometry has no obvious construction pathology and is suitable for the chosen next workflow.
On failure	A failed attachment leaves the prior molecule in place. Correct or discard any subsequent pending chemistry transaction. Do not treat failed cleanup as permission to accept an invalid graph or arbitrary pose.
Boundary	Fragment growth is not combinatorial design, synthesis planning, global docking, or proof that the new group is energetically favored.

Generic fragment-template staging and attachment use `fragment.stage` and `fragment.attach`; individual atom/bond edits and registered `designRoute.applyStep` operations remain available. For a registered, hash-pinned prospective route, an agent can stage a predefined product graph. `attachmentAtomId` is required only when that route declares designer-directed `spatialIntent`; the `SOS1` route does not:

```
await api.execute({action:"fragment.stage",
  args:{smiles:"c1ccncc1", name:"pyridyl", attachmentIndex:0}})
await api.execute({action:"fragment.attach",
  args:{attachedToAtomId:anchorId}})

await api.execute({action:"designRoute.load",
  args:{routeId:"sos1-hit-only"}})
await api.execute({action:"view.setMode", args:{mode:"build"}})
await api.execute({action:"protein.prepare", args:{
  pH:7.4, histidine:"auto", repairMissingHeavy:true,
  ligandPolicy:"ccd", waterPolicy:"retain", gapPolicy:"cap"}})
await api.execute({action:"pose.captureReference",
  args:{mode:"propagate"}})
await api.execute({action:"designRoute.applyStep",
  args:{stepId:"scaffold-rewrite"}})
await api.execute({action:"designRoute.inspect", args:{}})
```

Registered routes constrain prospective inputs and permitted edits; they are not retrospective campaign ledgers.

B.6 Relax a structure, run bounded dynamics, and inspect motion

Status. Current and experimental methods with different scopes.

The question. I have made a structure. Does its geometry survive relaxation, and what moves during a short calculation?

Choose the method from the scientific question, not from a generic expectation that every engine answers the same thing. Molarium exposes several numerical pathways whose energies are not interchangeable. Record the selected model, solvent, constraints, and protocol before interpreting the result.

Question	Method	What it provides	Principal boundary
Small-molecule cleanup or short force-field motion	RDKit	Energy, geometry optimization, and short dynamics using the RDKit force-field path.	The reported 0.1 fs dynamics path is a workbench calculation, not a production trajectory.
Sage energy, relaxation, or bounded single-system MD	Direct We-bGPU	Browser-GPU energy, forces, geometry, and saved frames from the numeric Sage system.	Experimental; f32; direct dynamics capped at 5,000 steps; no PBC or PME.
Many replicas of one topology	WebGPU ensemble	Homogeneous replica dynamics with energies, frames, timing, and constraint diagnostics.	Experimental; at most 512 atoms per replica, 1,024 replicas, and 100,000 steps; nonperiodic all-pairs model.
Electronic energy or geometry for an eligible neutral molecule	ANI-2x	Eight-model ANI-2x ensemble energy, spread, analytical forces, and optional optimized coordinates.	Experimental; connected neutral singlets with explicit H; H/C/N/O/S/F/Cl only; at most 96 atoms.
Reference calculation inside another workflow	OpenMM 8.2 Reference	Double-precision internal parameterization, scoring, relaxation, and validation path.	Internal subsystem, not a selectable general Simulate method.

Human workflow

1. Finish any pending chemistry. Confirm that the molecule is eligible for the chosen engine and, where required, can be completely parameterized.
2. In **Simulate**, choose the job and compatible method. Set the step count, vacuum or OBC2/ACE implicit solvent, X-H constraints where supported, saved frames, and ensemble options.
3. Run once. Inspect the engine or model identity, force field, charge model, solvent, constraints, initial and final energy, convergence or diagnostic status, elapsed time, and all warnings.
4. For dynamics, step through saved frames or replicas and inspect the region relevant to the design question. Display alignment does not overwrite raw calculation coordinates.
5. Review or export coordinates only when the result passes the declared acceptance check. Coordinate-producing jobs immediately update the current coordinates when they complete, and selecting or playing a saved frame writes that frame into the current molecule. Export the starting coordinates first when reversibility matters. Current-frame XYZ is available; saved frames are a bounded sample, and the workbench has no generic full-trajectory-file export.

Interpreting the model

Automatic Sage parameterization uses deterministic Gasteiger charges rather than AM1-BCC or NAGL. Unsupported elements, invalid chemistry, or missing required parameters fail instead of silently dropping terms. The exposed workflows are nonperiodic and do not provide PME, a barostat, explicit-solvent setup, generic production protein simulation, or binding free energy.

Agent skill contract: `relax_or_simulate`

Contract element	Requirement
Goal	Use a declared numerical model to test local geometry or bounded motion and retain enough metadata to interpret the result.
Preconditions	No chemistry transaction is pending; method eligibility and complete parameterization are established; an acceptance observable is specified.
Inputs	Job; method; steps; solvent; constraints; saved frames; fixed or movable atoms where supported; ensemble settings.
Procedure	Validate eligibility; record model and protocol; export the starting coordinates when reversibility matters; run one compatible job; inspect energies, diagnostics, and frames.
State mutation	Energy jobs do not move the molecule. Successful coordinate-producing jobs immediately replace current coordinates, and selecting a saved frame also writes current coordinates; raw frames remain calculation records.

Continued on next page

Contract element	Requirement
Evidence	Engine/model version and hashes where recorded, force field, charge model, solvent, constraints, timestep, energies, convergence/diagnostics, timing, frames, and warnings.
Success	The worker returns a finite, internally valid result and the predefined structural check is satisfied within the method’s declared scope.
On failure	Keep the prior graph and coordinates; record the returned error; correct eligibility, options, or assets before retrying.
Boundary	A minimized pose or short stable trace is not an affinity estimate, equilibrium ensemble, free-energy calculation, or production trajectory.

The current public API exposes a bounded Design optimizer:

```
await api.execute({action:"view.setMode", args:{mode:"build"}})
await api.execute({action:"optimization.run",
  args:{method:"induced-fit-webgpu"}})
await api.execute({action:"session.inspect",
  args:{scope:"pocket", includeCoordinates:true, maximumAtoms:500}})
```

General Simulate jobs use `calculation.run`; the action accepts only the installed engine/job combinations and the same bounded options as the interface. Selecting a returned frame is explicit because it writes those coordinates into the current molecule:

```
await api.execute({action:"calculation.run", args:{
  job:"dynamics", method:"stormm",
  options:{stormmSystem:"current", steps:1000,
    savedFrameCount:20, replicaCount:32}}})
await api.execute({action:"calculation.selectFrame", args:{index:19}})
```

B.7 Explore conformations with Conformer Arena

Status. Experimental current surface.

The question. What other conformations can this molecule adopt, and how sensitive is the answer to the generation and refinement method?

Conformer search is an alternative-generation problem. Molarium separates seed generation, physical refinement, clustering, comparison, and selection so that a low-energy-looking structure is not confused with proof that the ensemble is complete or Boltzmann weighted.

Human workflow

1. Start from a chemically finished small molecule with explicit hydrogens and a sensible protonation state. Recenter the structure before tight STORMM positional comparisons, because fixed-point spatial precision degrades far from the working origin.
2. Choose **WebGPU ensemble** and **Conformer search**. Set the requested conformer count, effort, clustering cutoff, solvent/constraint options, and whether to open the comparison arena.
3. Molarium generates deterministic RDKit conformer seeds, up to the implemented cap of 256. The STORMM protocol then performs staged relaxation, heating at 600 K, cooling through 450 K and 300 K, and final relaxation across homogeneous replicas.
4. Inspect the returned ensemble before selecting a representative: energy range, distinct RMSD clusters, cluster populations, recurring torsions, ring states, constraint diagnostics, and obvious duplicates or failures.
5. Use Conformer Arena to compare supported pathways on the same molecule and settings. Ask whether the methods discover the same basins, not merely whether they rank one shared conformer similarly.
6. Select a conformer only after inspection. Selection changes the displayed/current coordinates. Export the ensemble as SDF when it fits V2000 record limits; retain method and clustering metadata separately.

What does the result establish?

The workflow shows which basins the declared finite search found and how the supported methods behaved on that search. The anneal/minimize protocol is a search heuristic, so replica counts and cluster frequencies are not thermo-

dynamic populations. Failure to find a conformation is not evidence that it is inaccessible, and energy differences from different force fields are not directly comparable.

Agent skill contract: `explore_conformers`

Contract element	Requirement
Goal	Generate, refine, cluster, and compare a finite set of conformational alternatives for one molecular topology.
Preconditions	Chemistry and protonation are finalized; molecule is eligible; intended use and comparison metric are stated.
Inputs	Conformer count; effort; cluster cutoff; seed; solvent and constraints; arena choice; acceptance or diversity criteria.
Procedure	Generate RDKit seeds; run declared refinement protocol; cluster and inspect; compare basins in Arena when requested; select or export without rewriting raw records.
State mutation	Search produces an ensemble and raw calculation evidence. Selecting a frame or conformer writes its coordinates into the current molecule.
Evidence	Seed and method identity, requested and returned counts, energies, cluster assignments, diagnostics, timings, and exported ensemble where applicable.
Success	A finite set of valid, distinct candidates is produced and its diversity is sufficient for the stated downstream purpose.
On failure	Report parameterization, eligibility, replica, overflow, or pair-work limits; reduce scope or escalate rather than hiding missing replicas.
Boundary	Not exhaustive conformational sampling, a Boltzmann ensemble, solution free energies, or proof of bioactive-conformation probability.

Conformer search and Arena use `calculation.run`; result selection uses `calculation.selectConformer`. Arena axes, filtering, and sorting are presentation controls exposed through `calculation.setConformerView`; they do not create new conformers:

```
await api.execute({action:"calculation.run", args:{
  job:"conformers", method:"stormm",
  options:{conformerCount:32, conformerEffort:"standard",
    conformerClusterRms:0.75, conformerArena:true}}})
await api.execute({action:"calculation.selectConformer", args:{rank:0}})
```

B.8 Predict a short single-chain protein structure

Status. Experimental current surface; connected mode only.

The question. Can Molarium create a protein structure from sequence, and is this the co-folding workflow?

This is not currently co-folding. The implemented workflow predicts one protein entity from one sequence. It does not accept a ligand for joint protein-ligand prediction, does not predict a multimer, and does not maintain a starting ligand pose. Calling it co-folding would overstate the present capability.

The Fold Protein panel combines a configured remote MSA service with local OpenFold ONNX inference. The sequence is transmitted to the selected MSA endpoint; model inference then runs in the browser, preferring WebGPU and falling back to WASM. Local Lab disables the external MSA path.

Human workflow

1. Enter one amino-acid sequence using the standard residue alphabet or X. The current maximum length is 128 residues, using 64- or 128-residue feature buckets.
2. Inspect and accept the configured MSA endpoint and network boundary. First use may require approximately 540 MB of model assets.
3. Run and monitor MSA and inference progress. The current browser model executes three recycle passes; this count is not user-configurable. Cancel aborts the active MSA/prediction controller.
4. On success, Molarium loads the prediction as the current molecule. Inspect chain completeness, residue mapping, gross geometry, and suitability for the downstream question before replacing other work.
5. Export the predicted structure as PDB if it must persist. Preparation and local relaxation are separate decisions.

The current path does not use templates or perform Amber relaxation. It creates no server-side job ledger. Fetch, archive, feature, model, or inference failure produces no partial prediction. A returned structure is a model output, not experimental evidence or a prepared simulation system.

Agent skill contract: `predict_protein_structure`

Contract element	Requirement
Goal	Predict coordinates for one short protein sequence using the declared MSA service and local OpenFold model.
Preconditions	Connected mode; configured MSA endpoint; sequence length at most 128; network and model-asset use accepted.
Inputs	One sequence and MSA endpoint; the current model uses three recycle passes.
Procedure	Validate sequence; disclose external transmission; obtain and validate MSA; run the matching ONNX bucket; inspect result; export before replacing it.
State mutation	A successful prediction replaces the current molecule. Failure does not load a partial structure.
Evidence	Input sequence and endpoint, model/bucket and executed recycle count, progress/failure record, prediction metadata, and exported PDB where required.
Success	A complete finite prediction is returned for the declared input and passes basic structural inspection for the intended next step.
On failure	Report MSA, archive, rate-limit, maintenance, timeout, feature, asset, or inference failure; do not fabricate a partial structure.
Boundary	No ligand co-folding, multimer prediction, templates, Amber relaxation, experimental validation, or production preparation.

The workflow is exposed as `protein.predict`; `protein.cancelPrediction` aborts the active request. The MSA endpoint remains an explicit network boundary rather than a hidden service call:

```
await api.execute({action:"protein.predict", args:{
  sequence:fastaText, msaEndpoint:"https://msa.example/api"}})
// while active:
await api.execute({action:"protein.cancelPrediction", args:{}})
```

B.9 Maintain an experimental pose while changing ligand chemistry

Status. Experimental current surface.

The question. I trust the experimental pose of my starting ligand. How can I search for a plausible pose of an edited analogue without discarding what I already know?

Reference-guided docking is a local analogue pose-maintenance workflow. It preserves stable atom identities and selected interactions from a prepared reference complex, searches the degrees of freedom introduced by an edit, rejects infeasible candidates, and ranks distinct feasible poses in a rigid local receptor site. It is intentionally narrower than global docking.

Human workflow

1. Prepare and inspect the reference protein-ligand complex. In **Design**, capture the reference before editing. The capture stores ligand lineage, receptor atoms within 8 Å, reference contacts, selected hydrogen bonds, settings, and provenance.
2. Use the default **Pose propagation** mode to map surviving reference atoms and seed the edited region. Use **Expert selected core** only when you can justify at least three connected, non-collinear heavy atoms as a fixed reference.
3. Edit and finish the analogue. Review remapped or unavailable contacts, transformed ring regions, atom mapping, and any constraints before starting the search.
4. Run the constrained search. Molarium verifies receptor and contact integrity, releases eligible transformed rings, generates deterministic feature and torsion seeds, freshly parameterizes the edited ligand, relaxes feasible candidates, and scores the distinct results.
5. Inspect the ranked candidates, not only rank 1. Compare mapped-core displacement, preserved or replaced interactions, ligand strain, clashes, ring and side-chain choices, and the reasons candidates were rejected.

6. Apply one candidate explicitly. Candidate generation does not change the authoritative molecule. Application rechecks topology and receptor integrity before replacing coordinates. Export the labbook as JSON and human-readable Markdown/notes.

What is being scored?

The ranker uses a rigid receptor site within 8 Å, Lennard-Jones and Coulomb ligand-receptor cross terms with relative dielectric 4, and relative vacuum ligand strain. It omits general receptor relaxation, water displacement, entropy, binding free energy, and broad macrocycle or fused-ring concerted search. The separate induced-fit Design optimizer is not part of this ranker and is not an affinity calculation.

Negative results are results. An invalid core, altered receptor, unmappable required atom, unavailable contact feature, parameterization failure, or absence of feasible candidates produces an explicit failed or negative record. Molarium does not silently apply the least-bad pose.

Agent skill contract: `maintain_reference_pose`

Contract element	Requirement
Goal	Generate and review plausible local poses for one edited analogue while retaining explicit relationships to a trusted reference complex.
Preconditions	Prepared complex; trusted reference pose; reference captured before edit; chemistry finished; receptor unchanged; required contacts available.
Inputs	Pose-propagation or expert-core mode; selected core or contacts; cleanup/contact policy; candidate count; analogue graph.
Procedure	Capture; edit; finish; verify mapping and receptor integrity; generate and score feasible candidates; inspect alternatives; apply one explicitly or retain a negative result.
State mutation	Capture and search create reference and candidate records. Only Apply replaces the current coordinates.
Evidence	Protocol/settings snapshot, hashes, atom mapping, contact decisions, constraints, rejected candidates, scores, selected pose, and chained labbook.
Success	At least one feasible distinct candidate survives the declared constraints and a human or agent can justify any applied selection from the recorded evidence.
On failure	Apply nothing; preserve the negative labbook; repair the reference, mapping, constraints, or chemistry before rerunning.
Boundary	Local analogue pose maintenance only—not global docking, automatic MCS pose transfer, receptor-ensemble docking, or affinity/free-energy prediction.

```
await api.execute({action:"pose.captureReference",
  args:{mode:"propagate"}})
// perform and finish the intended molecular-graph edit
await api.execute({action:"pose.refine",
  args:{searchChains:16}})
await api.execute({action:"pose.apply", args:{index:0}})
await api.execute({action:"session.inspect",
  args:{scope:"pocket", includeCoordinates:true, maximumAtoms:500}})
```

For a receptor-side-chain alternative, the required order is:

```
await api.execute({action:"pose.enumerateSidechainRotamers",
  args:{receptorAtomId:sidechainAtomId, maximumCandidates:32}})
await api.execute({action:"pose.applySidechainRotamer", args:{index:5}})
await api.execute({action:"pose.updateReceptorReference", args:{}})
await api.execute({action:"pose.refine", args:{searchChains:64}})
```

Enumeration itself remains non-mutating; the selected receptor coordinates are applied explicitly and then made the new reference before another ligand-pose search.

B.10 Record, replay, and hand off a design process

Status. Current action/replay and Design History surfaces.

The question. I have done something scientifically useful. What must I export so that another scientist or an agent can understand and reproduce it?

Molarium records several different kinds of evidence. They answer different questions and should not be treated as interchangeable. The live action audit says what was requested; a calculation or workflow record says what method ran and what it returned; an action script says how to replay completed public operations; and Design History represents a durable sequence of content-addressed states and decisions. The current workbench does not automatically combine every scientific record into one complete project archive.

Record	Question answered	Persistence and limit
Chemist Actions audit	What recognized public action was requested, with what arguments and outcome?	Session memory; serialized calls; bounded history, normally 500 entries; cleared with the scene unless exported through a script path.
Preparation or calculation record	What system, model, settings, energies, frames, warnings, or audit produced this result?	Attached to the in-memory state and available through workflow-specific reports or exports.
Docking labbook	How were mapping, feasibility, scoring, rejection, and selection handled?	Exportable JSON and Markdown/notes with chained hashes.
Designer action script	Which completed public actions should be replayed?	Exportable JSON; replay invokes the same guarded routes; not a complete UI or binary session snapshot.
Registered design route	What prospective, hash-pinned edit/evaluation sequence is permitted?	Validated route schema; distinct from retrospective history.
Design History campaign	What content-addressed structures, decisions, commits, and branches form the design history?	Live browser surface with locally persisted commits, branches, merges, decisions, integrity events, import, and export.

Replay workflow

Designer Moves import begins from a blank canvas and performs script-level validation of schema, recognized action names, binding order, and forbidden shortcut keys. Individual action arguments and molecular preconditions are checked when each step executes. Conversion from an audit includes only completed public actions and excludes campaign administration. A standalone script therefore begins with an explicit reproducible loader such as `session.loadStructure`, `session.loadIdentifier`, `session.loadFixture`, or `designRoute.load`; it cannot depend on an unrecorded starting scene.

1. Export any unsaved molecular or workflow records before importing a script; import intentionally clears the existing scene.
2. Create a Designer action script from completed recognized public actions. Failed actions remain audit evidence but are not silently converted into successful replay steps.
3. Import the validated script, restart it, and Play or step through it. Replay uses the same Chemist Actions routes, runtime validations, and workers as manual operation and produces a separate replay audit.
4. Compare the resulting graph, coordinates, decisions, and errors with the intended endpoint. Export the replay audit and scientific records needed by the recipient.
5. For a registered story or route, verify schema, referenced content, and SHA-256 before execution. Treat a hash chain as integrity evidence, not an authenticated author identity or proof of scientific truth.

Replay provides play/pause, previous/next-step review, stable captions, and a separate audit. Failed audit entries remain evidence but are not converted silently into successful replay steps.

Live Design History

Design History is a live browser feature. `campaign.create` creates a locally persisted campaign and atomically commits the visible molecule. `campaign.commitCurrent` stores a complete graph-and-coordinate snapshot together with completed public actions since the prior commit. Branch switching restores the exact head molecule and refuses to proceed when the visible molecular state is dirty. Merge records the currently visible result with two parents; it is not an automatic structural merge. Decisions attach disposition, rationale, reason codes, and optional evidence to a commit. Verification recomputes content hashes, the append-only event chain, commit relationships, and branch heads derived from that chain.

Campaign JSON and the active branch are stored in browser IndexedDB. Reload restores the graph and coordinates at

the active branch head; it does not restore parameter tables, docking candidates, calculation frames, view state, or the complete workbench session. `campaign.import` verifies the canonical JSON and requires a restorable graph/coordinate snapshot before mutating live state; `campaign.export` returns that canonical JSON. Closing a campaign clears the active pointer without deleting its commits. A live commit can attach a script of completed molecular Chemist Actions; the complete molecular snapshot remains authoritative. Campaign-management actions are excluded from molecule replay scripts so replay cannot recursively administer its own history.

Choose the export that matches the handoff

Export	Carries	Does not carry
XYZ current frame	Displayed coordinates	Full topology metadata, history, or provenance
SDF conformer ensemble	Exportable conformer graph and coordinates	Unrelated session state or complete search history
PDB protein	Imported or predicted protein coordinates and supported annotations	Complete application or preparation provenance
Preparation JSON	Policies, issues, audit, and readiness status	A production-readiness guarantee
Docking JSON/-Markdown	Settings, mapping, candidates, failures, hashes, and selection	Author authentication or binding free energy
Action script JSON	Public actions and bindings	Arbitrary code, direct coordinates, or complete view state
PNG or share URL	A visual snapshot or current/registered URL	A reusable molecule or serialized live session

Agent skill contract: `export_and_handoff`

Contract element	Requirement
Goal	Preserve the smallest complete set of molecular, procedural, numerical, and decision records needed by the next scientist or agent.
Preconditions	The authoritative result and its intended downstream use are identified; unsaved state has not yet been cleared or replaced.
Inputs	Recipient and purpose; required molecular format; action/replay needs; calculation, preparation, docking, or campaign evidence to retain.
Procedure	Inventory records; export reusable molecular state and method evidence; create an action script when replay is required; commit decision history when project continuity is required; verify hashes and replay; describe missing pieces explicitly.
State mutation	Export is non-mutating. Script import is destructive to the unsaved scene and must follow export of work that must be retained.
Evidence	Exported files, schema and version identifiers, hashes where recorded, replay audit, and a statement of assumptions and unresolved failures.
Success	The recipient can identify the starting state, reproduce guarded actions where supported, inspect the method and result, and distinguish accepted decisions from candidates.
On failure	Stop on schema, action, binding, hash, or referenced-content mismatch. Do not describe a partial replay as equivalent to a completed audit.
Boundary	No complete project bundle or automatic capture of browser-native transport, calculation binaries, or every transient view gesture. An action script is not a substitute for the authoritative snapshot.

```
await api.execute({action:"campaign.create", args:{
  campaignId:"sos1-local-design", title:"SOS1 local design",
  initialCommitMessage:"Register 50VE/AXE starting state"}})
// chemist or agent performs guarded molecular actions
await api.execute({action:"campaign.commitCurrent", args:{
  message:"Accept Phe890-out design state", tags:["prospective"]}})
await api.execute({action:"campaign.recordDecision", args:{
  disposition:"progressed",
  rationale:"No new clashes after pocket-state search"}})
await api.execute({action:"campaign.verify", args:{}})
```

B.11 How skill contracts, API actions, and Design History fit together

Status. Current conceptual and executable relationship.

The question. How does a written agent contract become an authorized action and then a durable scientific decision?

The three layers serve different purposes and should not be collapsed:

1. **The skill contract defines scientific intent.** It states preconditions, permissible operations, required evidence, success, failure, and escalation. Here these names are proposed narrative operating contracts stored with the manuscript. They are not installed runtime skills, Chemist Actions, schemas, or executable enforcement.
2. **The Chemist Actions API defines executable authority.** `api.describe()` lists installed action names and bounded arguments. Calls are validated, serialized through one queue, and audited with sequence, status, result or error, and duration. An agent cannot gain extra molecular authority by phrasing a request differently.
3. **Design History defines durable meaning over time.** It commits complete molecular states and can link them to an explicitly partial public-action script, branches, decisions, evidence, actors, and append-only integrity records. The snapshot answers “what state was accepted?”; the action script answers “which guarded operations were captured and can be replayed?”; and the decision answers “why was it accepted?”

The practical agent pattern is:

```
const manifest = api.describe();           // discover installed authority
await api.execute({action:"session.inspect", args:{scope:"all"}})
await api.execute({action:"geometry.setInternalCoordinate", args:{
  atomIds:[a,b,c,d], value:172.0, moveConnected:true}})
await api.execute({action:"session.inspect", args:{scope:"all"}})
await api.execute({action:"campaign.commitCurrent",
  args:{message:"Accepted state after declared check"}})
await api.execute({action:"campaign.recordDecision",
  args:{disposition:"progressed", rationale:"..."}})
await api.execute({action:"campaign.verify", args:{}})
```

The manifest is the executable authority: a narrative skill contract cannot add an action, relax argument validation, or bypass a molecular precondition. Scientific review remains necessary even when an operation is exposed to both interface and agent.

Handoff rule. Export the structure when the recipient needs coordinates; export the calculation record when the recipient must judge the method; export the Designer Moves script together with its required starting state when the recipient must replay operations; and commit to Design History when the recipient must recover branches, decisions, and accepted molecular states. Hashes detect alteration of the serialized record. They do not establish authorship, chemical equivalence, or scientific validity.

The appendix in one sentence. For every task: make the molecular state explicit, choose a method whose scope matches the question, inspect before applying a candidate, retain the evidence needed to reproduce the decision, and stop when the declared scientific boundary is crossed.

Capabilities deliberately outside these workflows

The current workbench does not provide missing-loop construction, membrane or general metal preparation, covalent-ligand parameterization, periodic explicit solvent, PME, production-scale trajectories, binding free energy, global docking, general remote CUDA execution, GFN/xTB, general combinatorial enumeration, or automatic campaign selection. Their absence is part of the operating contract rather than a prompt to approximate them silently.

This task-centered appendix intentionally omits maintainer file paths, source-line numbers, worker lifecycle details, JSON nesting limits, deployment internals, and the complete repository test list. Those remain valuable implementation and audit documentation, but they are not required for a scientist or agent to understand how to perform and judge the workflows above.